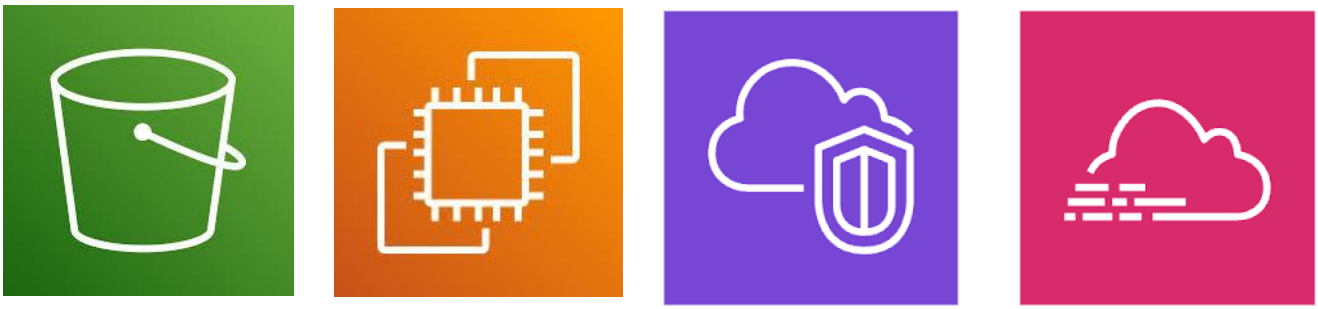
AHMED UMAR REHMAN

UNIVERSITY SPORTS ISSUING SYSTEM



Contents

1. Introduction 3

2. Objectives 3

3. Technologies Used 4

4. System Architecture 5

5. Implementation Details 6

5.1 Frontend Deployment (S3) 6

5.2 Backend Deployment (EC2) 7

5.3 Database Integration (DynamoDB) 8

5.4 User Login System 9

5.5 Booking System 9

5.6 Dashboard System 10

5.7 Email Notification System (SES) 11

6. Challenges Faced 12

7. Conclusion 12

8. Future Enhancements 12

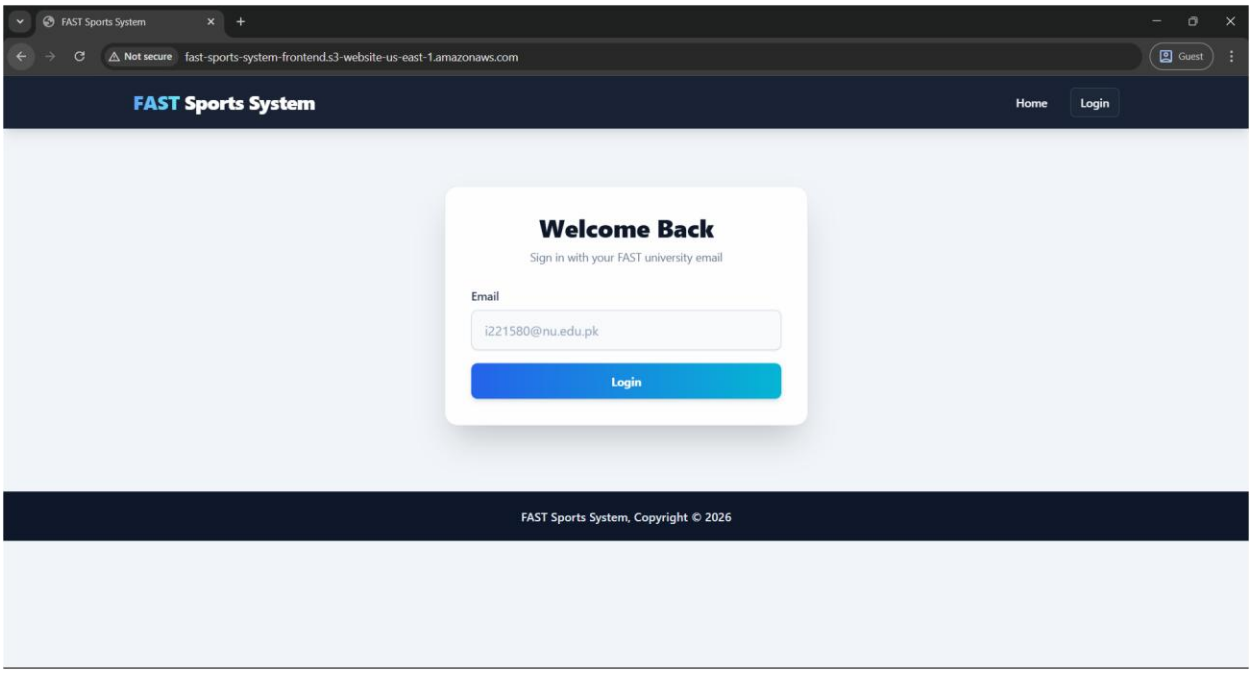
1. Introduction

The FAST Sports Booking System is a cloud-based web application developed to digitize the process of issuing sports equipment in a university environment. Traditionally, this process is handled manually, which can lead to inefficiencies and lack of tracking.

This system allows students to log in using their university email, view available sports items, book time slots, and receive email confirmations. The entire system is deployed on AWS, ensuring scalability, availability, and real-world cloud integration.

📸 **Screenshot:**

👉 Login Page



---

2. Objectives

- To develop a full-stack web application using cloud technologies
- To implement real-time booking and tracking system
- To store and manage data using a NoSQL cloud database
- To integrate email notifications for booking confirmation
- To deploy the application using AWS services

---

3. Technologies Used

Frontend

- React.js
- Tailwind CSS
- Hosted on Amazon S3

Backend

- Node.js (Express.js)
- Hosted on Amazon EC2

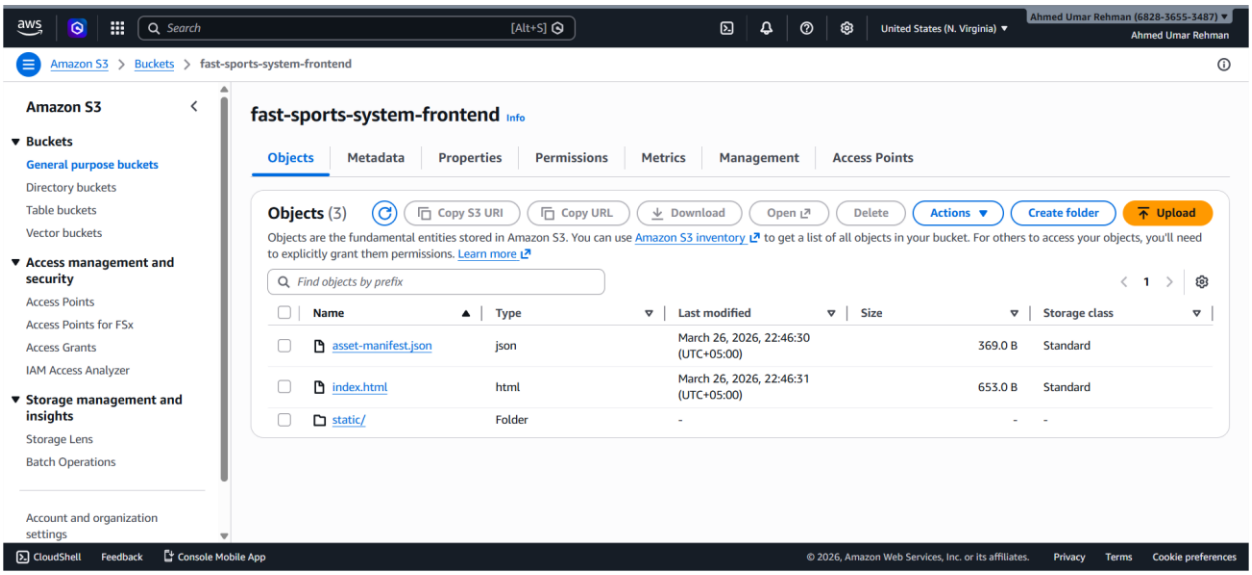
Database

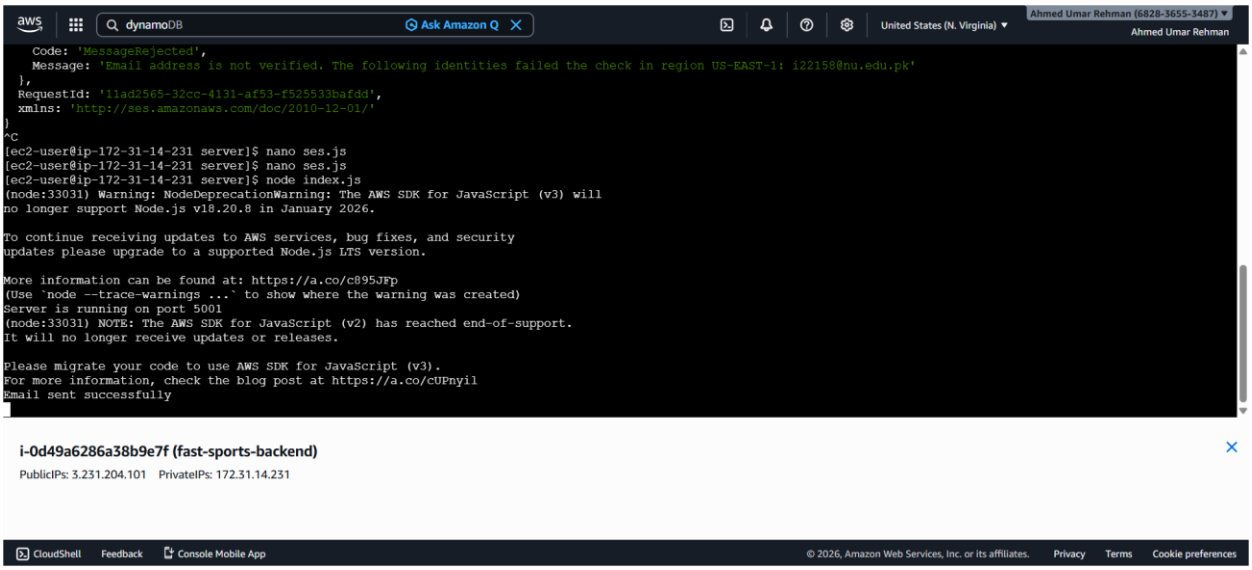
- Amazon DynamoDB

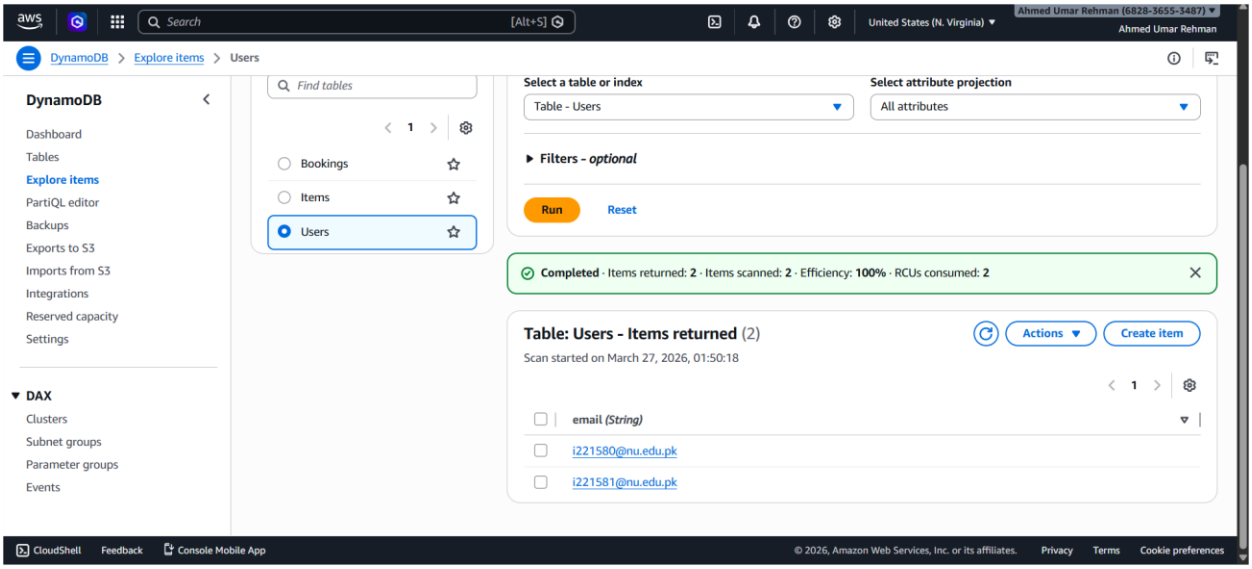
Email Service

- Amazon SES

📸 Screenshot:







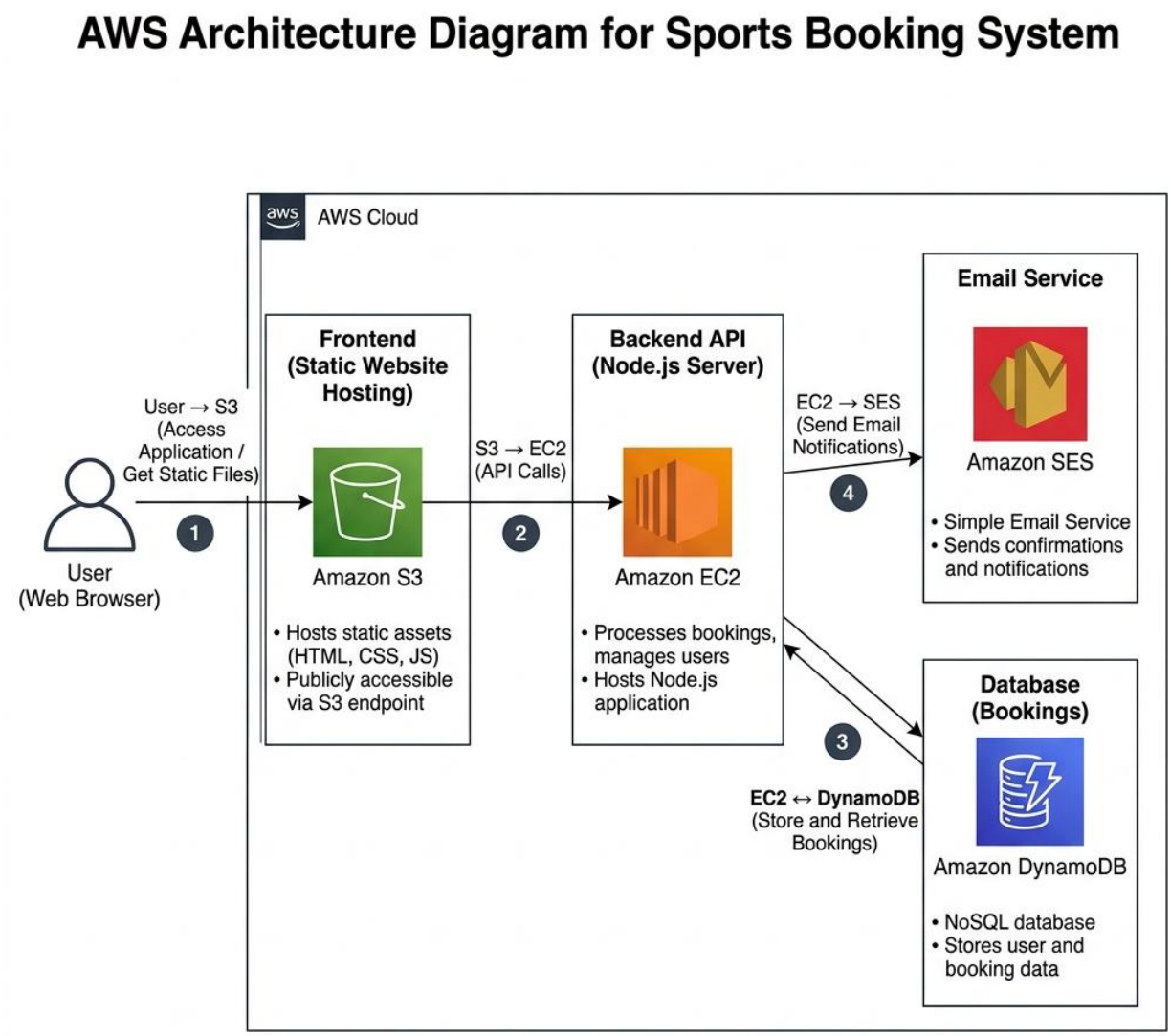
---

4. System Architecture

The system follows a client-server cloud architecture:

- The frontend is hosted on S3 as a static website
- The backend API is deployed on EC2
- DynamoDB is used for storing users and bookings
- SES is used to send email notifications

📸 **Screenshot :**



---

5. Implementation Details

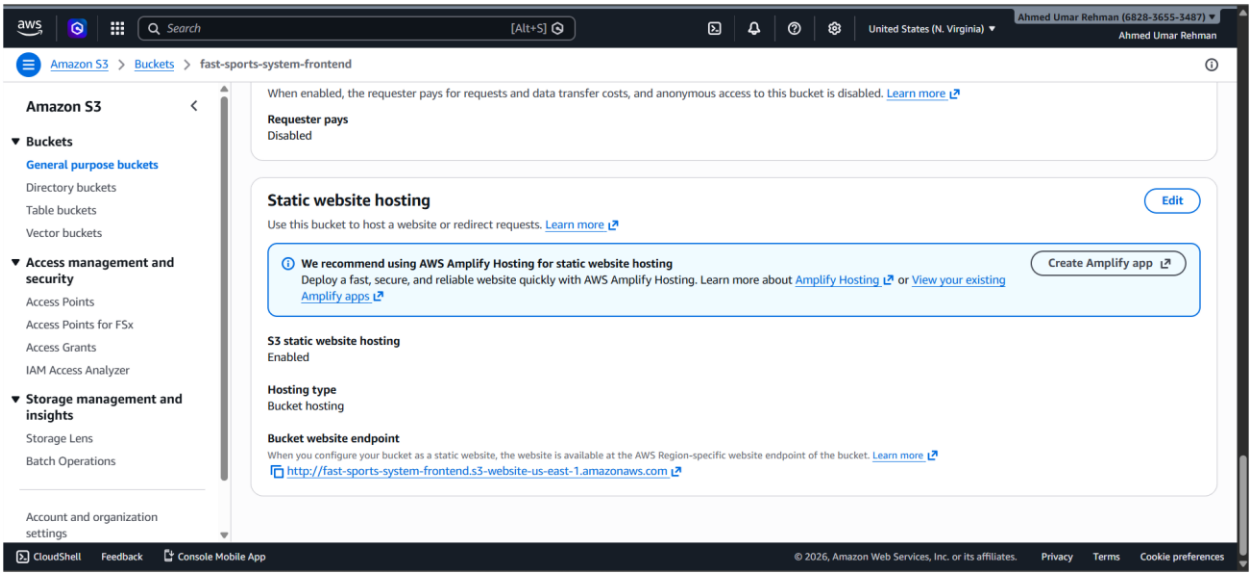
5.1 Frontend Deployment (S3)

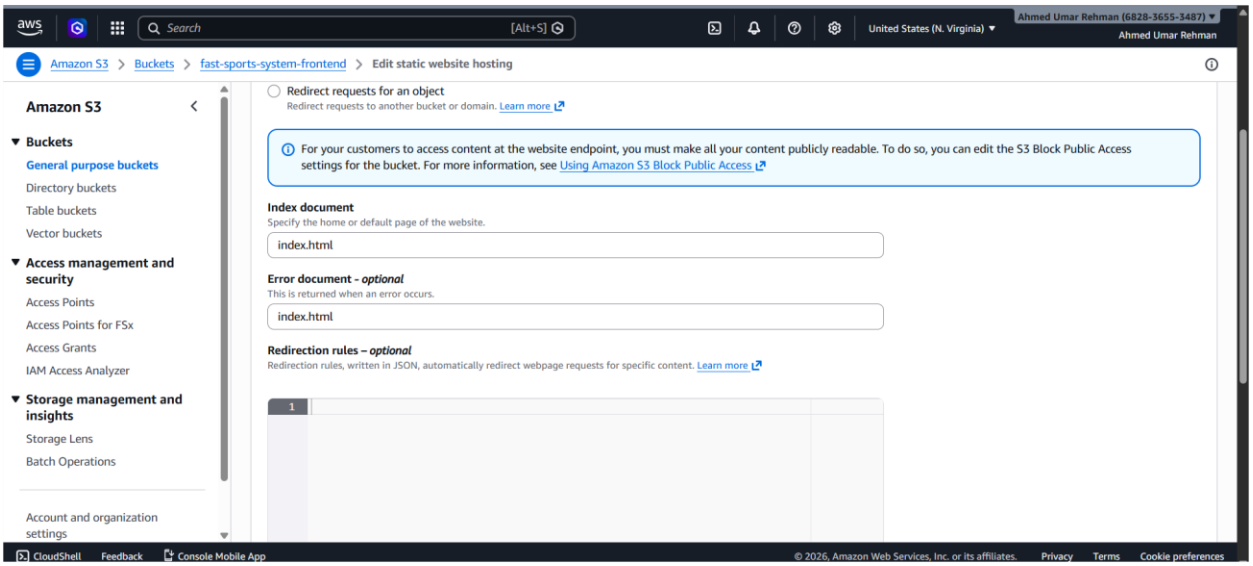
The frontend application was built using React and deployed on S3 using static website hosting.

Key configurations:

- Enabled static website hosting
- Set index.html as index document
- Fixed routing issue by setting error document to index.html

📸 Screenshot:





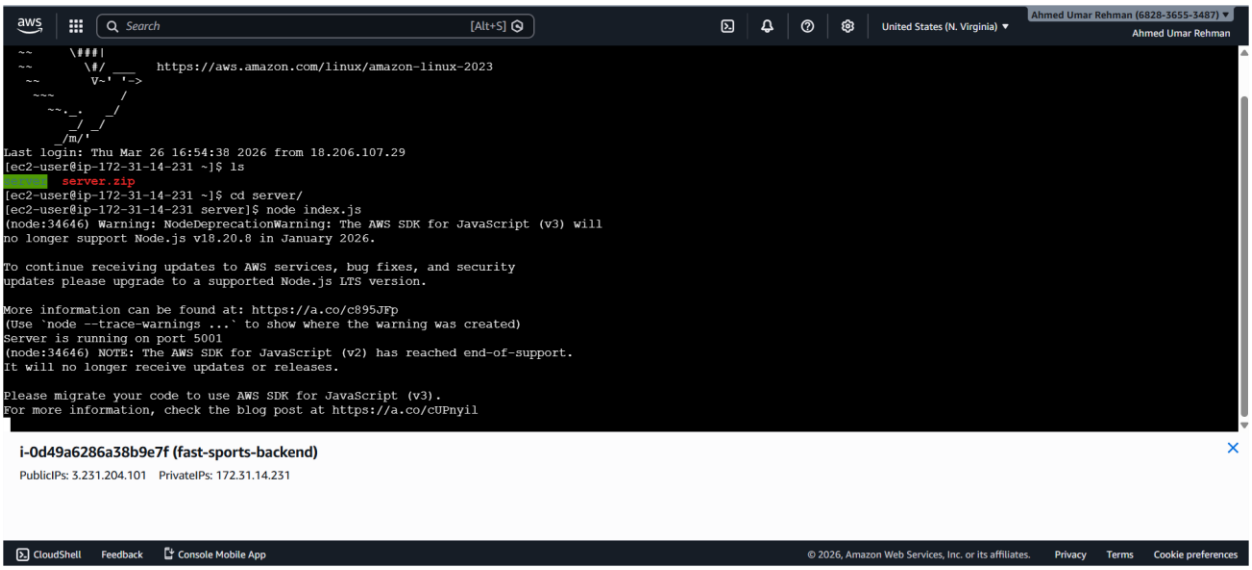
5.2 Backend Deployment (EC2)

The backend server was developed using Node.js and Express.js and deployed on an EC2 instance.

Steps:

- Created EC2 instance
- Installed Node.js
- Uploaded backend code
- Ran server on port 5001
- Configured security group

📸 **Screenshot:**



5.3 Database Integration (DynamoDB)

DynamoDB was used to replace in-memory storage and provide persistent data storage.

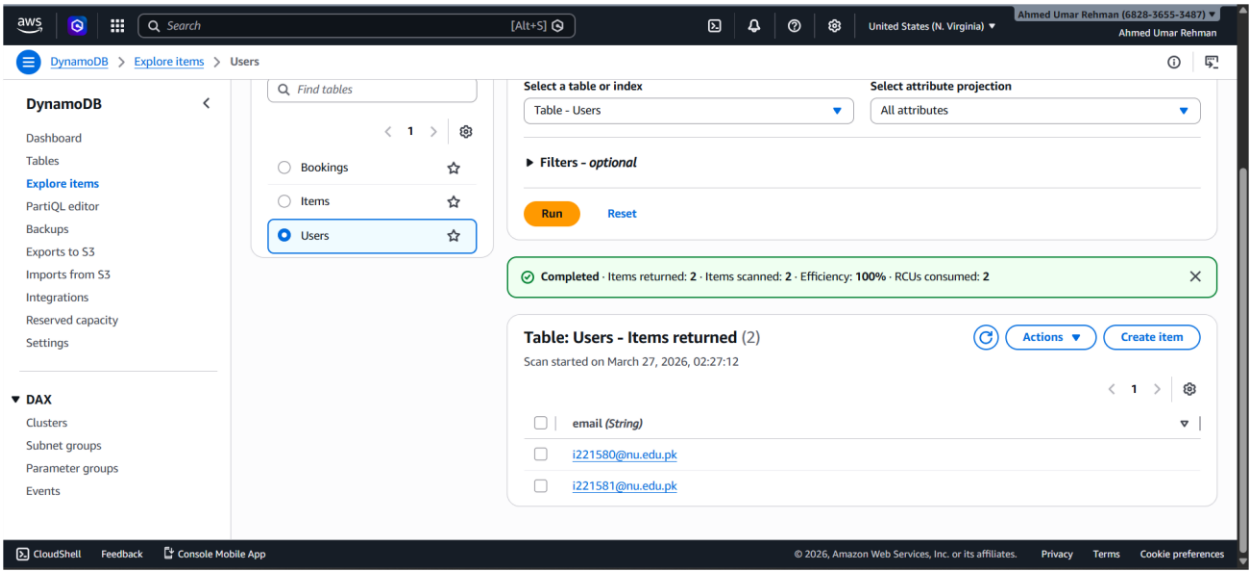
Tables created:

- Users
- Items
- Bookings

Features:

- High scalability
- Fast read/write performance
- No schema restrictions

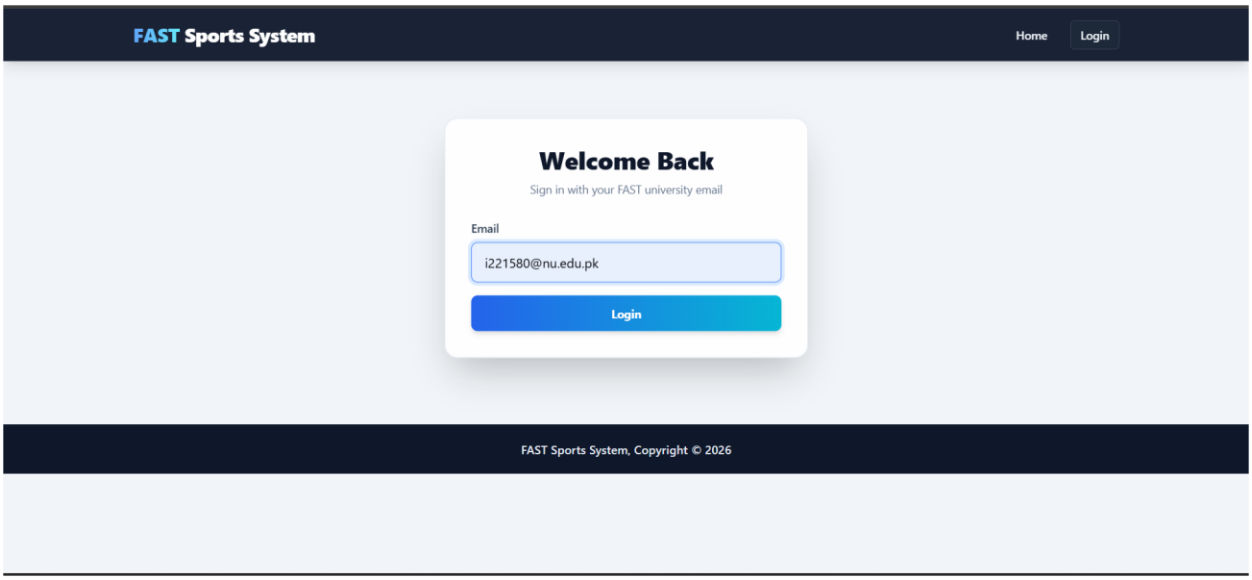
📸 **Screenshot:**



5.4 User Login System

Users log in using their university email. If the user does not exist in the database, a new entry is created automatically.

📸 **Screenshot:**



5.5 Booking System

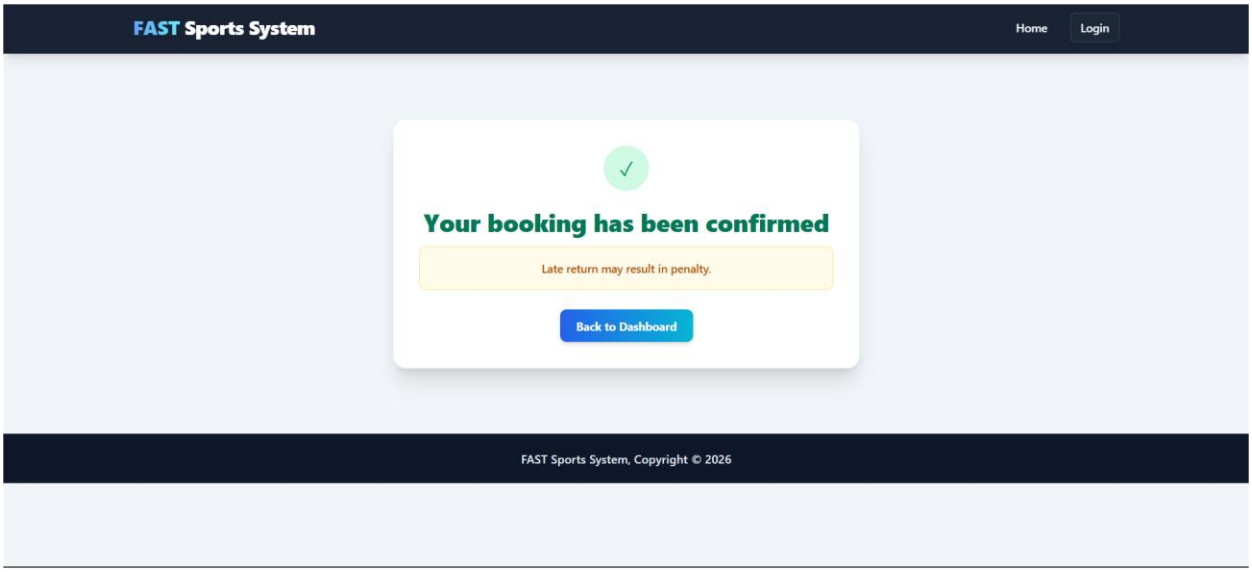
Users can:

- Select sports item
- Choose available time slot
- Submit booking request

Booking data includes:

- bookingId
- email
- itemName
- timeSlot
- bookingTime

📸 **Screenshot:**

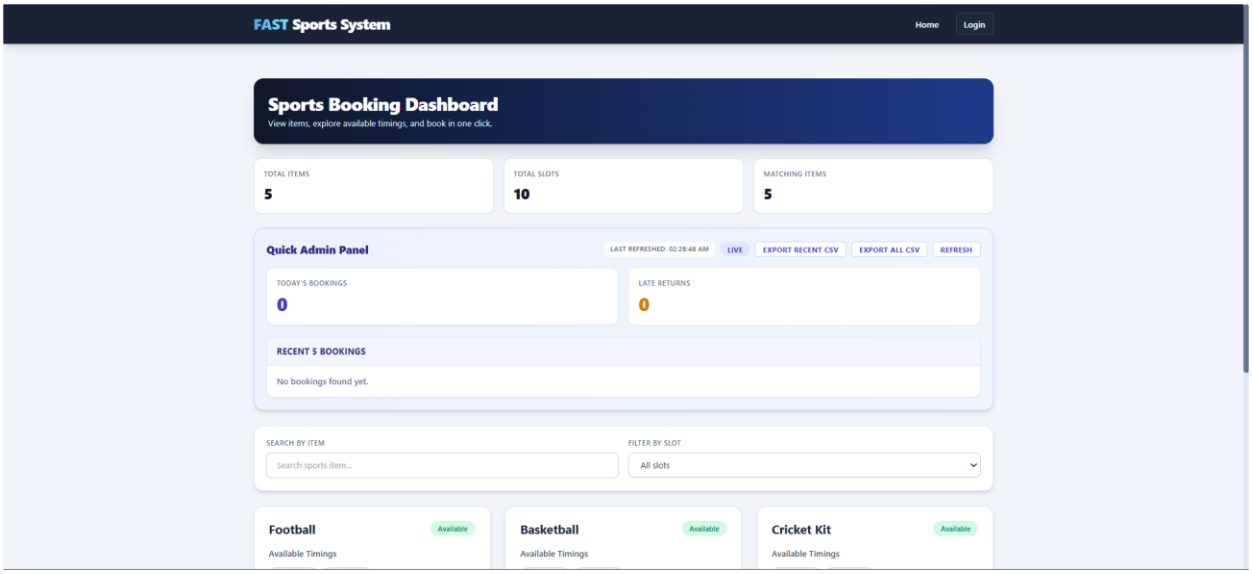


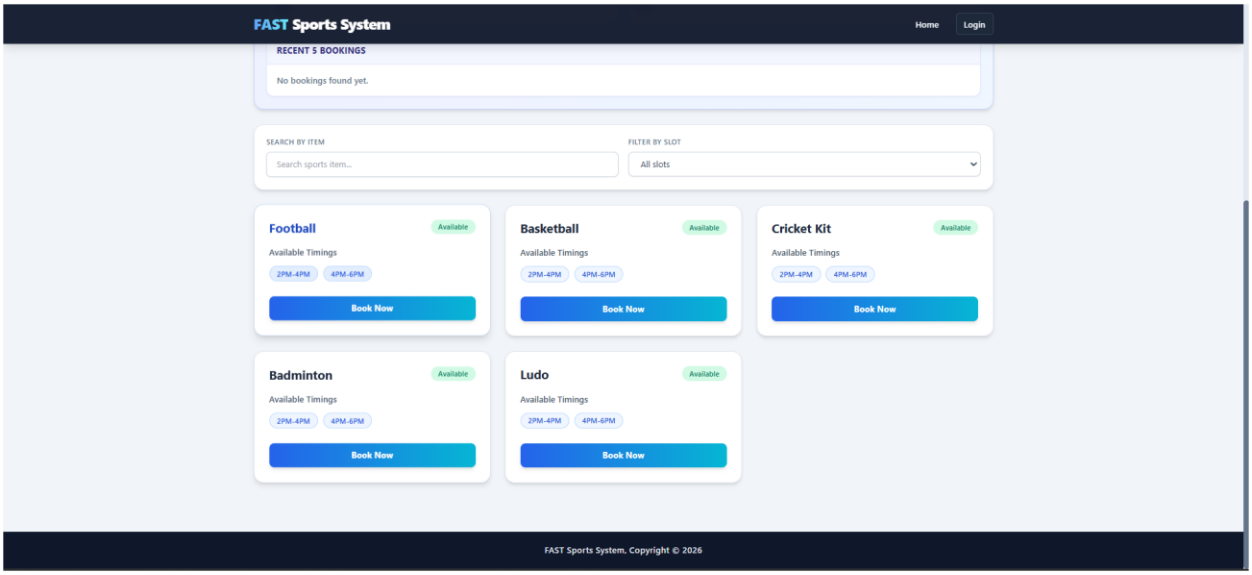
5.6 Dashboard System

The dashboard provides:

- List of available items
- Recent bookings
- Booking statistics
- Option to mark items as returned

📸 **Screenshot:**





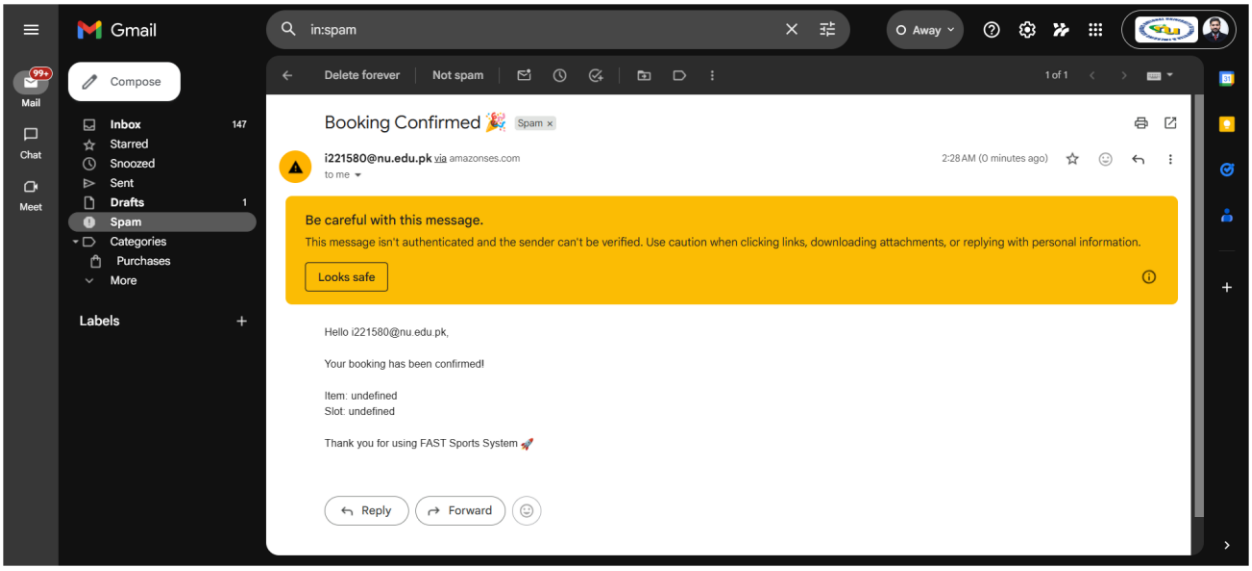
5.7 Email Notification System (SES)

Amazon SES is integrated with the backend to send email notifications when a booking is made.

Features:

- Sends booking confirmation email
- Uses verified email identities
- Works within SES sandbox environment

📸 **Screenshot:**



---

6. Challenges Faced

- Fixing S3 routing issue (404 on refresh)
- Handling DynamoDB key mismatches (case sensitivity)
- Configuring IAM roles for EC2 access
- Resolving SES permission and verification issues

---

7. Conclusion

This project demonstrates a complete cloud-based solution using AWS services. It showcases the integration of multiple services to build a scalable, reliable, and real-world application. The system successfully replaces manual processes with an automated cloud-based solution.

---

8. Future Enhancements

- Add authentication using JWT
- Move backend to serverless architecture
- Add analytics dashboard

- Enable SES production mode (remove sandbox limitation)